ICD: Graphics and Windowing System Architecture

Characterization and API Reference

Analysis of the ICD integrated circuit CAD program (1991–1992)

July 2026

1 Overview

ICD is a circa 1991–92 DOS integrated-circuit / schematic CAD program written in SVS Pascal 386 (32-bit flat-mode code running under a DOS extender), with a hand-written 80386 MASM assembly video driver underneath. There are no operating-system graphics services anywhere in the program — it owns the screen, writing directly into bank-switched SVGA framebuffer memory in the 16-color planar organization (VGA mode 12h style), per the design described in `DOCS/VIDEO.DOC` (based on Richard Wilton's *Programmer's Guide to PC & PS/2 Video Systems*, Microsoft Press, 1987). Everything above the framebuffer — fonts, line clipping, viewports, window frames, menus, buttons, and text-entry fields — is implemented from scratch.

The system is cleanly layered, with the Pascal/assembly boundary marked by `cexternal` declarations. The five layers are described below.

1.1 Layer 1 — Card-specific bank switching (`VIDEO/PIXEL.ASM`)

Each supported chipset gets a triple of small routines: `opn` (set the video mode), `cls` (card reset), and `seg` (select a 64 KB bank). Implemented families: `vga`, `vesa`, `vesab`, `pvga` (Western Digital / Paradise), `et4000` (Tseng), `ati`, `oti` (Oak), `trid` (Trident), and `gen` (user-supplied mode number). The driver is chosen at runtime from a label string such as `tseng1024x768x16` (see `DOCS/DRIVER.TXT`), and dispatch to the card code goes through function-pointer variables (`linep`, `linesp`, `linerp`, `blockp`, `setchrp`). Two pixel organizations are supported: planar (`*pl` routines) and packed (`*pk`, needed for ATI's nonstandard high-resolution mode).

1.2 Layer 2 — Assembly speed primitives (`VIDEO/DRAWA.ASM`, `DRAWB.ASM`, `PIXEL.ASM`)

Bresenham lines, filled blocks, and 16×16 bitmap character blitting, each in planar and packed variants, plus Cohen–Sutherland clipping (`clipline`). Notably, `VIDEO/DRAWA.PAS` and `DRAWB.PAS` contain the *same routines in portable Pascal* — reference implementations kept alongside the hand-optimized assembly, a useful bring-up and debugging discipline.

1.3 Layer 3 — Viewport graphics (ICDB.PAS)

The central abstraction is the `viewport` record (`DEFINE.PAS`, line 378): screen rectangle, real-world (design-space) rectangle, scale, multiplier, and a per-viewport clipping rectangle. All drawing calls take a viewport and *real* coordinates; `viewc` / `realc` transform between design space and screen space with rational (numerator/denominator) scaling. This layer adds arcs, circles, pies, grid snap, and proportional text, and — characteristic of the era — **rubber-banding by save-under**: every transient shape (cursor, markers, drag lines, zoom boxes, arcs) has a `set.../res...` pair that saves the pixels beneath it into a `linarr` buffer and restores them on undraw. There is no backing store or damage/repaint model; the screen is the only copy, and full redraws re-render the design from the display list.

1.4 Layer 4 — Windowing and menus (ICDC.PAS, ICDA.PAS)

A single-desktop windowing model built around the `winrec` record (`DEFINE.PAS`, line 476): whole viewport, client region, target viewport, top and right menu regions, active (drawing) area, plus lit/shadow/background colors used to draw 3D-beveled, Windows-style frames with a title bar and move buttons (`dispwframe`). `dispwin` is effectively a layout engine: it iteratively auto-arranges the button arrays into the top and right margins (`arrbutt` / `marginr` / `arrbutr`) until the menu fits, then places child viewports inside the parent with `plcwin`. The UI widget is the `butrec` "button" — a region with a string, active / disabled / alert states, per-state colors, screen-mode applicability sets, and layout-formatting flags — which doubles as an editable text field (`edtbut` / `getint` / `getrnm` parse numbers typed into a button in place).

1.5 Layer 5 — Input

Keyboard and timer are read via the BIOS in `IBMROT.ASM` (`kbdrdy` / `kbdinp` / `gettim`, plus an interrupt-vector hook). Pointer devices live in `POSITION/`: a Summagraphics Summasketch II tablet driven over raw COM1 serial I/O, or a Microsoft mouse via the INT 33h driver, unified behind `iniptr` / `updptr` with polling-based drag detection.

2 API Reference, base graphics through windowing

2.1 Card driver contract (per chipset, PIXEL.ASM)

Routine	Purpose
`<card>opn`	set the video mode, card-specific setup
`<card>cls`	card reset (does not restore old mode)
`<card>seg(s)`	select 64 KB video bank *s*

2.2 Physical screen layer (PIXEL.ASM / IBMROT.ASM — cexternal)

```
procedure iniscn(var maxx, maxy: integer; var s: labtyp);
                              { open display, select driver by label,
                                return resolution }
```

```
procedure resscn;             { restore former video mode }
function  getpix(x, y: integer): color;
                              { read pixel (screen coords) }
procedure setpix(x, y: integer; c: color);
                              { write pixel (screen coords) }
{ internal: pixadd (pixel address+bank calc), segvid, linbyt }
function  kbdrdy: boolean;    { keyboard char ready }
function  kbdinp: char;       { get keyboard char }
function  gettim: integer;    { system time }
```

2.3 Assembly drawing primitives (DRAWA.ASM / DRAWB.ASM)

Planar `pl` / packed `pk` variants sit behind the `linep...` dispatch pointers.

```
procedure clipline(...)       { Cohen-Sutherland line clip }
procedure line(vp, x1, y1, x2, y2, c)
                              { clipped Bresenham line in viewport }
procedure linesav(vp, x1, y1, x2, y2, c, var lines, var i)
                              { line, saving underlying pixels }
procedure linerst(vp, x1, y1, x2, y2, var lines, var i)
                              { restore pixels under a saved line }
procedure block(vp, x1, y1, x2, y2, c)
                              { filled rectangle }
procedure setchr(vp, x, y, ch, cl)
                              { blit 16x16 font glyph, clipped }
procedure inialpha; iniwidth;
                              { load bitmap font + proportional
                                width table }
```

2.4 Viewport graphics layer (ICDB.PAS — real-world coordinates)

```
{ coordinate transforms }
procedure viewc(var p: point; var vp: viewport);  { real -> screen }
procedure realc(var p: point; var vp: viewport);  { screen -> real }
function  scndist(d, s): integer;
function  realdist(d, s): integer;
procedure viewx / viewy(var vp; var x)
          { per-axis transform (driver level) }

{ shapes }
procedure box(vp, x1, y1, x2, y2, c);
procedure frame(vp, x1, y1, x2, y2, ...);
procedure arcc / arcr(...)       { arc by center / by radius }
procedure arcsavc / arcrstc(...) { arc with save-under / restore }
procedure pier(xc, yc, r, c, cr);  { filled circle/pie }
procedure con(vp, ...);            { connector dot }
```

```
procedure bliner / bboxr(...)      { broken (dashed) line / box }
procedure dogrid(xl, yl, c);       { dot grid }
procedure snapto(var x, y);        { grid snap }

{ rubber-band set/undraw pairs (save-under) }
setcur/rescur    setmrk/resmrk      setline/resline
setbox/resbox    setcircle/rescircle setarc/resarc
settcur/restcur  setrlm/resrlm      setzoom/reszoom
setasp           setend

{ text }
procedure plcchr(x, y, c, f, b, p, l, r);  { proportional char }
procedure plcstr(x, y, s, l, f, ...);      { proportional string }
procedure vchar(x, y, c, s, cl, ...);
          { scaled vector char (rotatable, IBMROT support) }

{ buttons / messages }
procedure plcbut(b: buttyp);
procedure updbut(b: buttyp);
procedure butact / butina / butuna / butalt / butdlt(b);
          { state changes }
function  onbutton / inbutton(b): boolean;  { hit testing }
procedure plcmsg(m: msgtyp; c: color);      { status message }
function  inactive / intarget(p: point): boolean;
          { region classification of a click }
```

2.5 Window management layer (ICDC.PAS / ICDA.PAS)

```
procedure plcwin(master, child: viewport; x1, y1, x2, y2);
          { place child viewport in master }
procedure dispwframe(wp: winptr; s: ttlstr);
          { draw beveled frame, title, move buttons,
            compute client area }
procedure dispwin;
          { full window layout: menus, target, active area,
            buttons, content }
procedure arrbutt / marginr / arrbutr / tfit / adjlin;
          { automatic button/menu layout }
procedure edit / edtbut(var b: butrec);
          { in-button text editing }
procedure getint / getrnm(var b, var n, var err);
          { parse number from button field }
procedure intstr / realstr(n; var s);
          { number -> string formatting }
procedure redraw; newview(x, y, s); dispsht; dispcell;
          { repaint pipeline }
procedure zoomin; zoomout; pan; back; bound;
```

```
          { view navigation }
procedure movcur(x, y); cruler; zruler; updrul; updcps;
          { cursor + rulers + coordinate readout }
```

2.6 Pointer/input layer (POSITION/)

```
procedure iniptr(var s: labtyp);
          { select+init device by label (summasii / msmouse=n) }
procedure updptr;
          { poll device, move cursor, dispatch drag/clicks }
{ device level: inisummasii/readsummasii,
                inimsmouse/readmsmouse }
{ raw serial (asm): iniaux, getaux, putaux, stsaux;
  mouse: inims, readmot, readsts }
```

3 Summary

ICD is a textbook early-1990s direct-to-metal GUI: a runtime-selected bank-switching chipset driver under assembly Bresenham/blit primitives, a real-coordinate viewport abstraction with save-under rubber-banding instead of backing store, and a self-contained windowing layer whose window frames, auto-layout menus, and editable button widgets anticipate much of what applications would later get from Microsoft Windows — all in roughly 700 KB of Pascal plus about 320 KB of assembly.